



Car Counting System

Machine Learning Project - 78

Submitted by:

Ahmed Mostafa Gomaa – 23P0375

Abdelrahman Mohamed Bahig – 23P0390

Supervised by Dr. Alaa Hamed





Table of Contents

Abstract	2
1. Introduction.....	2
2. Problem Definition.....	3
2.1. Objective	3
2.2. Constraints.....	3
2.3. Input and Output.....	3
3. System Overview	3
4. Dataset Description.....	4
4.1 Training Dataset	4
4.2 Dataset Processing.....	4
5. Training Pipeline: Car Classification.....	6
5.1 Feature Extraction	6
5.2 Model Training	6
5.3 Model Evaluation.....	7
5.4 Model Export.....	7
6. Inference & Counting Pipeline.....	8
6.1 Motion-Based Detection (Unsupervised).....	8
6.2 Contour Extraction and Filtering.....	8
6.3 Car Classification	8
6.4 Object Tracking	8
6.5 Counting Logic.....	9
7. Experimental Results	9
8. Project Structure	10
9. Reproducibility Notes	10
10. Conclusion.....	10
11. Future Work.....	11



Abstract

Traffic monitoring and vehicle counting are essential components of intelligent transportation systems. In this project, a complete **car counting system** is developed using **classical computer vision techniques combined with supervised machine learning**, without relying on pretrained deep learning models. The proposed system integrates motion-based object detection, Histogram of Oriented Gradients (HOG) feature extraction, and a Support Vector Machine (SVM) classifier to distinguish **cars from non-car objects**. Detected cars are tracked across video frames and counted when crossing a predefined virtual line. Experimental results on multiple real traffic videos demonstrate that the system can reliably count cars under typical traffic conditions while remaining computationally efficient and fully compliant with course constraints.

1. Introduction

Vehicle counting from traffic videos is a fundamental problem in computer vision with applications in traffic flow analysis, congestion management, and urban planning. While modern approaches frequently rely on deep learning and pretrained object detectors (e.g., YOLO, Faster R-CNN), such methods require large labeled datasets and significant computational resources.

The objective of this project is to design and implement a **fully self-trained car counting pipeline** using **classical computer vision and machine learning techniques only**. The system focuses exclusively on **passenger cars**, treating all other objects (buses, trucks, motorcycles, pedestrians, and noise) as non-target classes.



2. Problem Definition

2.1. Objective

To automatically count **cars only** in traffic videos recorded from static cameras.

2.2. Constraints

- No pretrained object detection or classification models
- No deep neural networks
- Training must be performed from scratch using a ready dataset
- Focus strictly on **car vs non-car** classification

2.3. Input and Output

- **Input:** Traffic videos captured from fixed viewpoints
- **Output:**
 - Annotated videos with bounding boxes, object IDs, and car count
 - Final numerical car count per video

3. System Overview

The system is divided into **two main pipelines**, implemented in separate Jupyter notebooks:

1. Training Pipeline

Dataset → Feature Extraction (HOG) → SVM Training → Model Export

2. Inference & Counting Pipeline

Video → Motion Detection → Classification → Tracking → Counting

The integration between the two pipelines is achieved through serialized model and configuration files.



4. Dataset Description

4.1 Training Dataset

The project uses the **Traffic Vehicles Object Detection** dataset from Kaggle, which contains:

- Real traffic images
- YOLO-format annotations
- Multiple object classes including *car*, *bus*, *truck*, and *two-wheeler*

Only the **car class** is treated as a positive label. All other classes are grouped into a single **non-car** category.

4.2 Dataset Processing

YOLO annotations are converted into pixel coordinates to crop objects from images.

Each crop is resized to a fixed resolution and stored in:

dataset/

└─ car/

└─ non_car/

This results in a clean, ready-to-use dataset for supervised binary classification.

```
=====
DATASET SANITY CHECK
=====
```

```
Car images:      4,116
```

```
Non-car images: 5,037
```

```
Total images:   9,153
```

```
Class distribution:
```

```
- Car:          45.0%
```

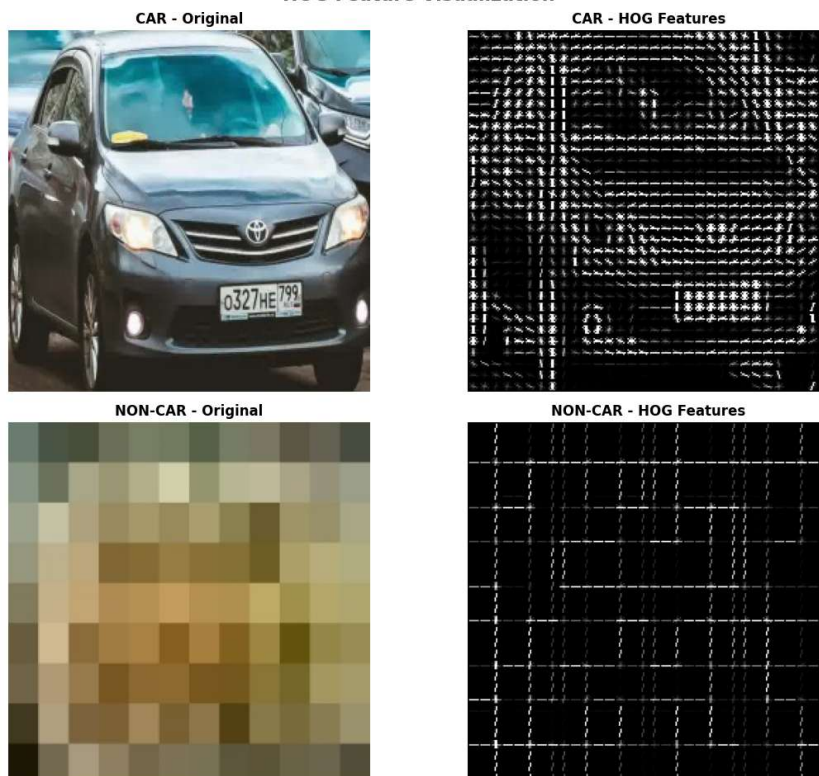
```
- Non-car:      55.0%
```

```
=====
```

Sample Images from Dataset



HOG Feature Visualization





5. Training Pipeline: Car Classification

5.1 Feature Extraction

Each cropped image undergoes the following preprocessing steps:

1. Resize to **64 × 64 pixels**
2. Convert to grayscale
3. Extract **Histogram of Oriented Gradients (HOG)** features

HOG Parameters:

- Orientations: 9
- Pixels per cell: (8, 8)
- Cells per block: (2, 2)

HOG was chosen due to its strong performance in capturing object shape and edge structure, which is well-suited for car appearance modeling.

5.2 Model Training

A **linear Support Vector Machine (SVM)** is used for classification.

Model Configuration:

- Kernel: Linear
- Regularization parameter: $C = 1.0$
- Probability estimates enabled

The dataset is split into:

- 80% training
- 20% testing (stratified split)

```
=====
TRAIN/TEST SPLIT
=====

Split ratio: 80% train / 20% test
Random state: 42

Training set:
- Total samples: 7322
- Cars (y=1): 3293
- Non-cars (y=0): 4029

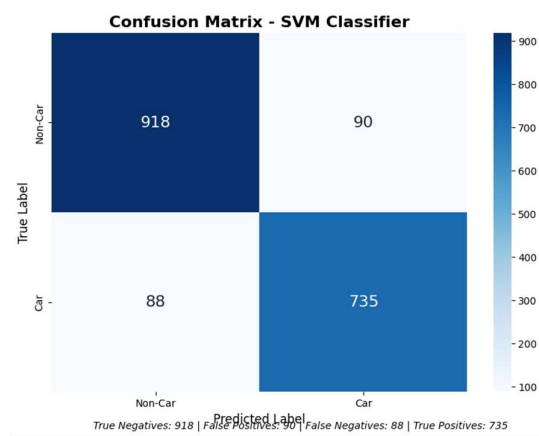
Test set:
- Total samples: 1831
- Cars (y=1): 823
- Non-cars (y=0): 1008
=====
```

5.3 Model Evaluation

Evaluation metrics include:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix

These metrics assess the classifier's ability to distinguish cars from non-car objects under diverse traffic conditions.



MODEL EVALUATION RESULTS				
Performance Metrics:				
Accuracy:	0.9028	(90.28%)		
Precision:	0.8909	(89.09%)		
Recall:	0.8931	(89.31%)		
F1-Score:	0.8920	(89.20%)		
Detailed Classification Report:				
	precision	recall	f1-score	support
Non-Car	0.91	0.91	0.91	1008
Car	0.89	0.89	0.89	823
accuracy			0.90	1831
macro avg	0.90	0.90	0.90	1831
weighted avg	0.90	0.90	0.90	1831

5.4 Model Export

To ensure consistency during inference:

- The trained SVM model is saved as `car_classifier_svm.joblib`
- All preprocessing and feature parameters are saved as `car_classifier_config.joblib`

This guarantees that inference uses the **exact same feature extraction settings** as training.



6. Inference & Counting Pipeline

6.1 Motion-Based Detection (Unsupervised)

Foreground objects are extracted using **Background Subtraction (MOG2)**:

- Adaptive background modeling
- Shadow suppression
- Morphological operations (open/close/dilate)

This step is fully **unsupervised** and identifies moving regions in the scene.

6.2 Contour Extraction and Filtering

Contours are extracted from the cleaned foreground mask and filtered using geometric heuristics:

- Minimum and maximum area thresholds
- Aspect ratio constraints
- Noise suppression

This stage reduces false positives before classification.

6.3 Car Classification

For each remaining candidate:

1. Crop bounding box from original frame
2. Resize to training crop size
3. Extract HOG features
4. Apply SVM classifier

A confidence threshold (default 0.6) is applied to ensure robust car detection.

6.4 Object Tracking

A **centroid-based tracker** assigns unique IDs to objects across frames by minimizing Euclidean distance between detections. This tracking approach is lightweight and suitable for real-time scenarios but may degrade under heavy occlusion.



6.5 Counting Logic

A horizontal virtual counting line is placed at a configurable vertical position in the frame.

A car is counted **only once** if:

- It is classified as a car
- Its tracked center crosses the counting line
- Its ID has not been counted before

This prevents double counting.

7. Experimental Results

The system was tested on **7 real traffic videos**, producing:

- Annotated output videos with bounding boxes and IDs
- Final car counts per video

Observed Strengths

- Robust performance under moderate traffic density
- Accurate car discrimination
- Low computational cost

Observed Limitations

- Occlusion between cars may break tracking
- Large perspective changes affect bounding box size
- Nighttime or extreme lighting conditions reduce accuracy



8. Project Structure

```
ML project - car counter|
├─ car_classifier_config.joblib
├─ Car_Classifier_SVM.ipynb
├─ car_classifier_svm.joblib
├─ dataset
│  └─ car
│  └─ non_car
├─ output
│  └─ car_count_1.mp4
│  └─ car_count_2.mp4
│  └─ car_count_3.mp4
│  └─ car_count_4.mp4
│  └─ car_count_5.mp4
│  └─ car_count_6.mp4
│  └─ car_count_7.mp4
│  └─ car_count_output.mp4
├─ README.md
├─ videos
│  └─ 1.mp4
│  └─ 2.mp4
│  └─ 3.mp4
│  └─ 4.mp4
│  └─ 5.mp4
│  └─ 6.mp4
│  └─ 7.mp4
└─ Video_Car_Counter.ipynb
```

9. Reproducibility Notes

- Re-running the crop-generation step requires restoring the YOLO dataset
- Feature parameters must remain unchanged between training and inference

10. Conclusion

This project demonstrates that accurate car counting can be achieved using **classical computer vision and machine learning techniques** without pretrained models. By combining motion-based detection, HOG features, and SVM classification, the system achieves reliable performance while remaining computationally efficient and academically sound.



11. Future Work

Possible improvements include:

- Kalman filter–based tracking
- Adaptive counting lines for multi-lane roads
- Shadow removal enhancement
- Perspective-aware size normalization
- Comparison with deep-learning approaches (conceptually, not implemented)

12. References

1. Dalal, N., & Triggs, B. (2005). *Histograms of Oriented Gradients for Human Detection*. CVPR.
2. Cortes, C., & Vapnik, V. (1995). *Support-Vector Networks*. Machine Learning.
3. OpenCV Documentation — Background Subtraction
4. Scikit-learn Documentation — Support Vector Machines